

CS 218 - Design and Analysis of Algorithms:

Assignment 1

Total Marks - 80

Instructions. Please be concise, clear, and technically accurate. Describe all algorithms using pseudocode, followed by a brief discussion. Wherever applicable, draw upon the algorithmic techniques covered in class, and ensure that proofs are rigorous; lack of formality may result in mark deductions.

Question 1 [20 Marks]: Let T be a binary tree whose nodes store distinct numerical values. Recall that T is a **binary search tree** if and only if either (1) T is empty, or (2) T satisfies the following recursive conditions:

- The left subtree of T is a binary search tree.
- All values in the left subtree are smaller than the value at the root.
- The right subtree of T is a binary search tree.
- All values in the right subtree are larger than the value at the root.

Consider the following pair of operations on binary trees:

- **Rotate** an arbitrary node upward.

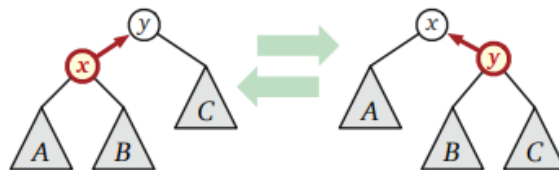


Figure 1: Rotation of an arbitrary node upward.

- **Swap** the left and right subtrees of an arbitrary node.

In both of these operations, some, all, or none of the subtrees A , B , and C could be empty.

(a) [5 Marks] Describe an algorithm to transform an *arbitrary* n -node binary tree with distinct node values into a binary search tree, using at most $O(n^2)$ rotations and swaps. Figure 3 shows a sequence of eight operations that transforms a five-node binary tree into a binary search tree.

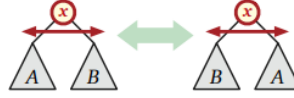


Figure 2: Swapping left and right subtrees.

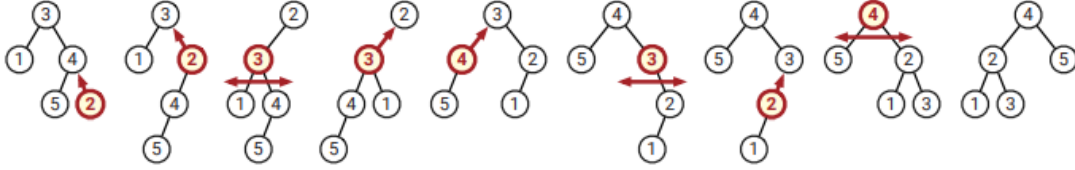


Figure 3: “Sorting” a binary tree: rotate 2, rotate 2, swap 3, rotate 3, rotate 4, swap 3, rotate 2, swap 4.

Your algorithm is not allowed to directly modify parent or child pointers, create new nodes, or delete old nodes; the *only* way to modify the tree is through rotations and swaps.

On the other hand, you may *compute* anything you like for free, as long as that computation does not modify the tree; the running time of your algorithm is *defined* to be the number of rotations and swaps that it performs.

(b) [10 Marks] Describe an algorithm to transform an arbitrary n -node binary tree into a binary search tree, using at most $O(n \log n)$ rotations and swaps.

(c) [5 Marks] Prove that any n -node *binary search* tree can be transformed into any other *binary search* tree with the same node values, using only $O(n)$ rotations (and *no swaps*).

(d) [**Bonus problem – 5 Marks:**] Either describe an algorithm to transform an arbitrary n -node binary tree into a binary search tree using only $O(n)$ rotations and swaps, or prove that no such algorithm is possible.

Question 2 [20 Marks]: Welcome to the **MAGICAL CANDY SHOP!** You sell **n** unique types of candy boxes (like, “**SUPERSOUR**,” “**CHOCOBLAST**,” etc.). To create these candies, there are exactly 26 TYPES OF MAGIC CHARMS available in the world (representing ‘A’ through ‘Z’).

Every candy box is defined by a specific composition of these charms. For example, a “**SUPERSOUR**” box requires exactly: 2 Charms of type ‘S’, 2 Charms of type ‘U’, 2 Charms of type ‘R’, 1 Charm each of ‘P’, ‘E’, ‘O’.

Let $S = \{s_1, s_2, \dots, s_n\}$ be the set of all candy flavors you sell. You initially ordered enough ingredients to manufacture **m copies** of every single flavor s_i in the set S . Thus, your total initial inventory of a specific charm c is:

$$\text{Total}(c) = m \times \sum_{i=1}^n \text{count}(s_i, c)$$

Right before the delivery truck arrived, a hungry dragon attacks and destroys the *entire shipment* for **exactly one** specific flavor, which we will call s_ℓ . Now, you have a pile of ingredients that was supposed to make m copies of all flavors except the one the dragon ate. Given m and all candy

box names, for each of $l \in \{1, 2, \dots, n\}$, your primary goal is to reconstruct at least one copy of the missing flavor s_ℓ or let us know if such a reconstruction is impossible.

(a) [5 Marks] Under what mathematical condition(s) would the reconstruction be physically impossible?

(b) [5 Marks] Assuming that the reconstruction of the missing flavor s_ℓ is possible, calculate the maximum integer k (where $0 \leq k < m$) such that you can:

- Sacrifice enough ingredients from the remaining stock to build **at least 1 copy** of the missing flavor s_ℓ .
- Still have enough ingredients left to build **k full copies** of all $n - 1$ other surviving flavors (s_i where $i \neq \ell$).

FORMALLY DISCUSS THE CORRECTNESS OF YOUR ANSWER.

(c) [5 Marks] Write the pseudocode to solve the problem for every possible scenario where a flavor goes missing (for all $\ell \in \{1, \dots, n\}$). Your solution should utilize the logic from Sub-problems 1 and 2. Use 2D array `count[n][26]`, where `count[j][i]` stores the number of times charm i appears in flavor j . **Note:** If you cannot even make *one* copy of the missing flavor, return the answer as -1.

(d) [5 Marks] Based on your pseudocode in Sub-problem 3:

- What is the time complexity to process a single query for a missing flavor ℓ ?
- What is the total time complexity for all n flavors?

Express your answer in terms of n (number of flavors) and $|\Sigma|$ (size of magic charms, which is 26 in this case). **Note:** Running time of your algorithm should be polynomial in n .

Question 3 [20 Marks]: Your new boss at the Apsara Pencil Factory asks you to design an algorithm to solve the following problem. Suppose you are given N pencils, each with one of c different colors, and a non-negative integer k . **How many different ways are there to choose a set of k pencils?** Two pencil sets are considered identical if they contain the same number of pencils of each color.

For example, suppose you have two **red** pencils, four **green** pencils, and one **blue** pencil. Then you can form exactly five different two-pencil sets (**RR**, **RG**, **RB**, **GG**, **GB**), exactly six different four-pencil sets (**RRGG**, **RRGB**, **RGGG**, **RGGB**, **GGGG**, **GGGB**), and exactly three different six-pencil sets (**RRGGGG**, **RRGGGB**, **RGGGGB**).

Input: An array `Pencils[1...c]` and an integer k , where `Pencils[i]` stores the number of pencils with color i .

Output: A single non-negative integer representing the number of distinct sets.

For example, given the input `Pencils = [2, 4, 1]` and $k = 2$, your algorithm should return the integer 5.

- (a)[10 Marks] Describe an algorithm to solve this problem that runs in $O(c \cdot k^2)$ time.
- (b)[10 Marks] Describe an algorithm to solve this problem that runs in $O(c \cdot k \log k)$ time.
- (c)[**Bonus problem – 5 Marks**]: Can you do better? Describe an algorithm that solves the same problem in $O(c \cdot k)$ time. (*Hint: Improve the algorithm in part (a).*)

Question 4 [20 Marks]: A student maintains a study plan consisting of a sequence of tasks arranged in a fixed order. Each task is assigned a positive integer *difficulty level*. For organizational purposes, the student wishes the difficulty levels to be *non-decreasing* from left to right.

The student is allowed to perform the following operation any number of times:

Select a task with difficulty w and replace it by two consecutive tasks with positive integer difficulties x and y such that $x + y = w$.

The relative order of all other tasks must remain unchanged.

For example, if $A = [1, 3, 2]$, the student may split the second task to obtain $[1, 1, 2, 2]$, which is non-decreasing.

For a given task sequence L , define its *adjustment cost*, denoted $\text{cost}(L)$, as the minimum number of operations required to transform L into a non-decreasing sequence. You may assume that such a transformation is always possible.

Over n weeks, the student records the difficulty levels

$$A = (A_1, A_2, \dots, A_n),$$

where $1 \leq A_i \leq B$ for a fixed bound B .

For any contiguous interval of weeks $[i, j]$, the corresponding task sequence L_{ij} is

$$(A_i, A_{i+1}, \dots, A_j).$$

Define

$$T(A) = \sum_{1 \leq i < j \leq n} \text{cost}(L_{ij}),$$

where the sum ranges over all non-empty contiguous task sequences extracted from A .

(a)[5 Marks] For a fixed task sequence L , design an optimal strategy to compute $\text{cost}(L)$. Prove rigorously that your strategy is both feasible and optimal. Additionally, derive a closed-form expression for $\text{cost}(L)$ that is linear in $|L|$.

(b)[5 Marks] Derive a closed-form expression for $T(A)$ whose evaluation requires $\mathcal{O}(n^2)$ time, and justify its correctness. (*Hint: Analyze the contribution of an ordered pair of indices (i, j) to $T(A)$.*)

(c)[10 Marks] Propose and justify a dynamic programming method to compute $T(A)$ asymptotically faster than $\mathcal{O}(n^2)$, exploiting the bound B . (*Hint: An $\mathcal{O}(n\sqrt{B})$ -time solution is possible.*)